



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI SCIENZE MATEMATICHE,
FISICHE E NATURALI

CORSO DI LAUREA IN INFORMATICA

Tesi di laurea triennale

Flussi di liquidità nella Lightning Network: Analisi di
un modello con bias regolabile

2024-2025

RELATORE:

Francesco Pasquale

CANDIDATO:

Emanuele Ventili

Indice

1	Introduzione	3
2	Modello e Problema	6
2.1	Modellazione formale della Lightning Network	6
2.1.1	Canali di pagamento e smart contract	7
2.2	Capacità e bilanci	7
2.3	Transazioni e instradamento	8
2.4	Fallimento del pagamento e asimmetria informativa	9
2.5	Dinamiche di traffico: uniformità vs. asimmetria	9
2.6	Proprietà topologiche e misure di centralità	10
2.6.1	Edge betweenness centrality	10
2.6.2	Cut-set e isolamento del Merchant	11
2.7	Stabilità della rete e tempo di fallimento	12
2.7.1	Stato del sistema e passeggiata aleatoria	12
2.7.2	Il tempo di fallimento τ	13
2.8	Meccanismi di Ribilanciamento: Il JIT Rebalancing	13
2.8.1	Conservazione della liquidità locale	14
2.8.2	Selezione Ottimale del Canale di Uscita	14
3	Metodologia e Modello di Simulazione	15
3.1	L'Algoritmo di Simulazione	16
3.2	Configurazione del Sistema e Ipotesi Iniziali	17
3.2.1	Inizializzazione dei Bilanci e Stato di Equilibrio	17
3.2.2	Quantizzazione dei Pagamenti	17
3.3	Modellazione del Traffico Asimmetrico	17
3.3.1	Descrizione dell'algoritmo di Rebalancing	19
3.3.2	Descrizione dell'algoritmo: PCN Simulation con Rebalancing	22
4	Simulazioni e Risultati	23
4.1	Ambiente di Sviluppo	23
4.2	Implementazione	23

4.3	Grafico τ in funzione di α	24
4.4	Grafico τ in funzione di k in scala log log	25
4.5	Analisi del trasferimento di ricchezza e del Sink Effect	26
4.6	Frequenza di fallimento degli archi in funzione di Betweenness Centrality e del Merchant bias	27
4.7	Grafico τ in funzione di k Nella Lightning Network	28
4.8	Effetti del Rebalance nella lightning Network	30
5	Conclusioni	32
5.1	Limiti e sviluppi futuri	33
	Elenco delle figure	35
	Elenco degli algoritmi	35

Capitolo 1

Introduzione

Negli ultimi anni, Bitcoin si è affermato come il principale sistema di pagamento decentralizzato al mondo. Tuttavia, la sua architettura presenta un limite strutturale significativo: la blockchain di Bitcoin è in grado di processare circa 7 transazioni al secondo, un numero molto basso per supportare un'adozione su larga scala come sistema di pagamento quotidiano. Per risolvere questa limitazione, nel 2016 Poon e Dryja hanno proposto la Lightning Network [1], un protocollo di secondo livello che consente di eseguire transazioni off-chain in modo istantaneo e a costo quasi nullo, sfruttando canali di pagamento bidirezionali aperti tra coppie di nodi. La Lightning Network può essere modellata come una Payment Channel Network (PCN): un grafo in cui i nodi rappresentano i partecipanti e gli archi rappresentano i canali di pagamento attivi. Ogni canale ha una capacità fissa, ovvero la quantità di bitcoin bloccata al momento dell'apertura, che viene ridistribuita tra i due endpoint ad ogni transazione. Questo approccio permette di effettuare pagamenti rapidi e privati senza coinvolgere ogni volta la blockchain, che interviene solo nelle fasi di apertura e chiusura del canale. Una delle sfide di questo modello riguarda la gestione della liquidità. A differenza della capacità totale dei canali, che è pubblica e nota a tutti i nodi della rete, i bilanci dei singoli endpoint (ovvero la suddivisione della liquidità tra i due endpoint di ciascun canale) sono informazioni private, note solamente ai due endpoint. Questa asimmetria informativa è la causa del problema più rilevante della Lightning Network: il fallimento dei pagamenti. Quando un nodo tenta di instradare un pagamento lungo un percorso, non ha la garanzia che i canali intermedi dispongono di liquidità sufficiente per continuare a instradare il pagamento. Se almeno un canale risulta insufficiente, il pagamento fallisce e il mittente deve calcolare un percorso alternativo.

Il problema si accentua in presenza di flussi di pagamento asimmetrici. Nella Lightning Network reale, i pagamenti non vengono distribuiti uniformemente tra tutti i nodi: esistono nodi che svolgono il ruolo di mercanti,

ad esempio exchange di criptovalute, piattaforme di e-commerce, fornitori di servizi digitali, che ricevono pagamenti con frequenza molto più alta rispetto a nodi ordinari. Questa asimmetria introduce una deriva nei bilanci dei canali connessi a questi nodi Merchant: la liquidità tende a finire unidirezionalmente verso di essi, portando rapidamente all'esaurimento dei canali in ingresso e rendendo i mercanti irraggiungibili anche quando la capacità totale della rete è ancora abbondante.

Questa tesi si propone di analizzare quantitativamente l'impatto di questa asimmetria sulla stabilità della rete, con particolare attenzione al tempo di primo fallimento τ , ovvero il numero di transazioni completate con successo prima che si verifichi il primo fallimento dato dall'esaurimento di un canale. Il lavoro prende come punto di partenza i risultati teorici del paper del relatore Francesco Pasquale e collaboratori [2], che nel modello a traffico uniforme dimostrano una crescita quadratica di τ rispetto al numero di nodi n e alla capacità dei canali k . Il contributo originale di questa tesi consiste nell'estensione di questo modello al caso con nodi Merchant, introducendo un parametro di bias α che regola la probabilità che la destinazione di ogni pagamento sia un nodo Merchant piuttosto che un nodo scelto casualmente.

Le simulazioni condotte mostrano che anche valori moderati di α determinano un crollo drastico di τ la crescita quadratica del modello uniforme cala a un regime sub-quadratico, e nel caso della topologia reale della Lightning Network la relazione diventa quasi lineare. Questo risultato indica che il semplice aumento della capacità dei canali non è sufficiente a garantire la stabilità della rete in presenza di traffico asimmetrico: il problema è strutturale, radicato nella topologia del grafo e nella distribuzione del traffico. Dato che il problema della stabilità sembra essere radicato nella distribuzione del traffico piuttosto che nella sola capacità totale, la tesi indaga anche l'efficacia di manovre di ribilanciamento. In particolare, viene analizzato se e in quale misura la capacità dei nodi Merchant di ridistribuire in maniera autonoma la propria liquidità possa posticipare il primo fallimento della rete in presenza di forti asimmetrie.

La tesi è organizzata come segue: Il capitolo 2 introduce il modello formale della Lightning Network, le definizioni di capacità, bilanci, transazioni

e tempo di fallimento, e descrive i meccanismi di ribilanciamento proposti. Il capitolo 3 presenta la metodologia di simulazione e gli algoritmi implementati. Il capitolo 4, espone i risultati sperimentali e le analisi quantitative ottenute su topologie a Clique e su uno snapshot reale della rete.

Capitolo 2

Modello e Problema

Questo capitolo introduce le fondamentali teorie alla base dello studio fatto, molte formalizzazioni e i risultati teorici riguardo lo studio del tempo di fallimento della Lightning Network sono stati presi e riassunti da me prendendo spunto dal paper del mio relatore Francesco Pasquale e suoi collaboratori [2]. Nella Sezione 2.1 viene richiamata brevemente la struttura del *Lightning Network* dal punto di vista della teoria dei grafi. Nelle Sezioni 2.2–2.4 viene definita capacità, bilanci, transazioni e il concetto di fallimento di un pagamento. La Sezione 2.5 introduce la differenza tra modello a traffico uniforme e modello a traffico asimmetrico (con nodi *Merchant*), che costituisce il contributo originale di questa tesi. Le Sezioni 2.6 e 2.7 discutono le proprietà topologiche dei grafi applicate all’oggetto di studio e formalizzano il concetto di primo fallimento τ . La sezione 2.8 propone una tecnica di ribilanciamento.

2.1 Modellazione formale della Lightning Network

La *Lightning Network* (LN) [1] è una *Payment Channel Network* (PCN): una struttura a grafo in cui i nodi rappresentano i partecipanti alla rete e gli archi rappresentano i canali di pagamento bidirezionali aperti tra di essi.

Formalmente, la rete viene modellata come un grafo non orientato e pesato $G = (V, E)$, dove V è l’insieme dei nodi (i partecipanti) ed E è l’insieme dei canali attivi. La *Lightning Network* non possiede un proprio registro distribuito autonomo: la liquidità che circola nella rete consiste in bitcoin reali, la cui proprietà è vincolata all’interno di *smart contract* multi-firma (2-of-2 *multi-sig*) registrati sulla blockchain di *Bitcoin*.

Il vantaggio del modello risiede nelle transazioni *off-chain*: invece di trasmettere ogni operazione alla rete *Bitcoin* per la validazione, processo lento e costoso, le parti aggiornano privatamente i propri bilanci tramite messaggi firmati crittograficamente. La blockchain principale interviene esclusivamen-

te nella fase iniziale anche detta fase di *opening* (blocco dei fondi) e nella fase finale anche detta fase di *closing* (registrazione dello stato finale).

2.1.1 Canali di pagamento e smart contract

L'apertura di un canale avviene tramite una transazione sulla blockchain che deposita i fondi in un indirizzo *multi-signature*: i bitcoin bloccati possono essere spesi solo se entrambi i partecipanti firmano congiuntamente l'operazione. Finché il canale rimane aperto, i due nodi si scambiano privatamente messaggi di aggiornamento che ridistribuiscono la proprietà di quei bitcoin all'interno del canale, senza mai utilizzare direttamente la blockchain. La sicurezza di questo meccanismo è garantita da protocolli crittografici — in particolare gli *Hash Time-Locked Contracts* (HTLC) — che rendono impossibile a uno dei due partecipanti truffare l'altro pubblicando uno stato di bilancio non aggiornato.

2.2 Capacità e bilanci

Ogni canale $e = \{u, v\} \in E$ ha un peso $c(e) \in \mathbb{N}$, detto *capacità del canale*. Questa capacità è **pubblica**, ovvero tutti i nodi della rete sanno quanti bitcoin sono stati bloccati in ogni canale. La capacità è suddivisa tra i due endpoint: definiamo il *bilancio del canale* come la coppia $\{b_e(u), b_e(v)\}$, dove $b_e(u)$ e $b_e(v)$ rappresentano i fondi posseduti rispettivamente da u e da v . È molto importante che la somma dei bilanci tra due nodi collegati da un canale sia uguale al bilancio del canale, ovvero:

$$b_e(u) + b_e(v) = c(e), \quad b_e(u), b_e(v) \in \{0, 1, \dots, c(e)\}. \quad (2.2.1)$$

A differenza della capacità, i bilanci sono **privati**: sono noti esclusivamente agli endpoint del canale u e v , non agli altri partecipanti della rete. Questa *asimmetria informativa* è una caratteristica fondamentale della *Lightning Network* e, come vedremo, è alla radice del problema dei fallimenti dei pagamenti.

Configurazione iniziale. Ai fini dell'analisi, viene assunto che al tempo $t = 0$ ogni canale si trovi in uno stato di perfetto equilibrio. Formalmente, viene quindi imposto

$$c(e) = 2k \quad \forall e \in E, \quad k \in \mathbb{N}, \quad b_e(u) = b_e(v) = k.$$

Questa scelta è fondamentale per isolare l'impatto delle dinamiche di traffico sulla stabilità della rete: partendo da una configurazione neutra, ogni spostamento verso lo sbilanciamento sarà associato esclusivamente al flusso di transazioni generato durante la simulazione, eliminando bias dati da una distribuzione iniziale sbilanciata tra gli endpoint.

Canale vuoto in una direzione. Un canale si dice *vuoto in una direzione* quando il suo bilancio minimo, definito come:

$$b_{\min}(e) = \min\{b_e(u), b_e(v)\} = 0, \quad (2.2.2)$$

ovvero quando uno dei due endpoint ha esaurito la propria quota di liquidità. In tale condizione diventa impossibile instradare fondi in quella direzione.

2.3 Transazioni e instradamento

Un pagamento tra due nodi adiacenti u e v di importo x consiste in un aggiornamento atomico del bilancio:

$$\begin{cases} b'_e(u) = b_e(u) - x \\ b'_e(v) = b_e(v) + x \end{cases} \quad (2.3.1)$$

se e solo se viene rispettata la condizione $x \leq b_e(u)$.

Nel caso di nodi non adiacenti, il pagamento $\text{pay}_P(u, v, x)$ viene instradato lungo un cammino $P = (u = u_0, u_1, \dots, u_h = v)$, aggiornando il bilancio di ogni canale $e_i = \{u_i, u_{i+1}\}$ lungo il percorso secondo la regola (2.3.1). La condizione necessaria affinché il pagamento abbia successo è che l'importo x non superi il bilancio disponibile in *ogni* singolo canale del percorso:

$$x \leq b_{e_i}(u_i) \quad \forall i = 0, \dots, h-1.$$

L'operazione è atomica: o tutti i bilanci lungo P vengono aggiornati, oppure nessuno viene aggiornato.

2.4 Fallimento del pagamento e asimmetria informativa

Mentre la topologia del grafo G e le capacità $c(\cdot)$ sono informazioni pubbliche, i bilanci $\{b_e(u), b_e(v)\}$ sono noti esclusivamente agli endpoint di ciascun canale. Dunque può accadere che un nodo mittente che vuole effettuare un pagamento selezioni un cammino P che non dispone della liquidità necessaria in almeno uno dei suoi archi:

$$\exists e_i \in P : b_{e_i}(u_i) < x.$$

In questo scenario si verifica un *fallimento del pagamento*: il mittente non è in grado di completare la transazione lungo P e deve ricalcolare un percorso alternativo se esiste.

2.5 Dinamiche di traffico: uniformità vs. asimmetria

Nella *Lightning Network* reale, i pagamenti non sono distribuiti uniformemente tra i nodi della rete: esistono nodi che svolgono il ruolo di *mercanti* — negozi online, exchange di criptovalute, piattaforme di servizi — che ricevono pagamenti con frequenza molto superiore rispetto ai nodi ordinari. Un utente qualsiasi, nel corso di una sessione, pagherà quasi certamente uno o più mercanti, non un nodo scelto a caso nella rete.

Questa tesi studia come la presenza di tali nodi influenzi la dinamica dei fallimenti dei pagamenti: in particolare, come cambia il tempo al primo fallimento τ quando la destinazione dei pagamenti non è uniforme ma concentrata su un sottoinsieme privilegiato di nodi.

- **Modello uniforme.** Sorgente e destinazione di ogni pagamento vengono scelte uniformemente a caso tra tutti i nodi. Le variazioni dei bilanci seguono una *passeggiata aleatoria simmetrica* (*unbiased random walk*): la probabilità di un incremento o decremento della liquidità su

un arco è la stessa in entrambe le direzioni. Il sistema gode di un meccanismo di auto-bilanciamento statistico, e il primo fallimento tarda a manifestarsi.

- **Modello con nodi Merchant.** La destinazione dei pagamenti segue una distribuzione *non uniforme*: un sottoinsieme $\mathcal{M} \subset V$ di nodi *Merchant* viene scelto come destinazione con probabilità molto più alta. Questo introduce una componente di *deriva* (*drift*) che spinge i bilanci dei canali connessi ai mercanti verso il limite di esaurimento, accelerando il fallimento in modo direttamente proporzionale all'intensità dell'asimmetria.

Questo fenomeno di drenaggio unidirezionale della liquidità verso i nodi Merchant verrà d'ora in avanti indicato come **Sink Effect**. A differenza di uno squilibrio temporaneo, il Sink Effect è strutturale: non si corregge spontaneamente al crescere del tempo, ma si accentua proporzionalmente all'intensità del bias α . La conseguenza pratica è che i canali del cut-set dei Merchant si svuotano progressivamente in modo unidirezionale, rendendo i Merchant irraggiungibili anche quando la liquidità totale della rete è ancora abbondante.

2.6 Proprietà topologiche e misure di centralità

La topologia del grafo G influenza l'efficienza e la resilienza della *Lightning Network*. Non tutti i canali contribuiscono nello stesso modo per l'instradamento dei pagamenti: la struttura della rete determina l'esistenza di canali *critici*, la cui posizione li rende passaggi obbligati per molti cammini minimi tra coppie di nodi. L'importanza di un arco non è quindi legata esclusivamente alla sua capacità, ma è determinata dalla sua *centralità* all'interno della topologia complessiva.

2.6.1 Edge betweenness centrality

La metrica più rilevante per l'analisi della saturazione dei canali è la *edge betweenness centrality*. Essa quantifica il carico di traffico potenziale che

pesa su un determinato arco, nell'ipotesi che i pagamenti vengano instradati seguendo i *cammini minimi* (*shortest paths*). Formalmente, per un arco $e \in E$:

$$g(e) = \sum_{s, t \in V} \frac{\sigma(s, t \mid e)}{\sigma(s, t)}, \quad (2.6.1)$$

dove $\sigma(s, t)$ è il numero totale di cammini minimi tra s e t , e $\sigma(s, t \mid e)$ è il numero di quei cammini che includono l'arco e .

Un valore elevato di $g(e)$ identifica un canale *critico* — un collo di bottiglia della rete soggetto a una frequenza di aggiornamento del bilancio superiore alla media. Nel modello uniforme, sono proprio questi archi i primi a esaurire la liquidità. L'introduzione di nodi *Merchant* modifica la distribuzione della betweenness: il traffico si concentra sui canali incidenti ai mercanti, accelerando drasticamente il processo di sbilanciamento indipendentemente dalla betweenness globale di quegli archi.

2.6.2 Cut-set e isolamento del Merchant

Per comprendere perché la rete possa smettere di funzionare avendo ancora liquidità complessiva disponibile, è utile introdurre il concetto di *cut-set*. Nella teoria dei grafi, un cut-set è un insieme di archi che, se rimossi, disconnette il grafo in due o più componenti. Nel contesto delle PCN, un arco non viene rimosso fisicamente, ma diventa *logicamente inattivo* in una direzione quando il suo bilancio raggiunge il valore critico $b_{\min}(e) = 0$.

Questo fenomeno è rilevante in presenza di nodi *Merchant*: poiché per costruzione attraggono la maggior parte delle transazioni, i canali che li collegano direttamente al resto della rete formano un cut-set critico. Il crollo del tempo di fallimento τ è quindi riconducibile alla saturazione di questo *cut-set*. Una volta che il cut-set critico è sbilanciato, il *Merchant* risulta irraggiungibile dal punto di vista dei pagamenti in entrata — anche se il resto del grafo mantiene bilanci perfettamente equilibrati.

2.7 Stabilità della rete e tempo di fallimento

Il cuore dell'analisi è determinare per quanto tempo la rete sia in grado di sostenere un flusso continuo di transazioni prima del primo fallimento. In un contesto reale, i nodi possono adottare strategie di manutenzione — come il *rebalancing* (spostamento manuale o automatizzato di fondi tra i propri canali) o l'apertura di nuovi canali — per prolungare l'operatività della rete. Il seguente argomento verrà trattato nei capitoli a seguire.

2.7.1 Stato del sistema e passeggiata aleatoria

Lo stato della rete al tempo t è la raccolta dei bilanci di tutti i canali attivi. Per un singolo canale e di capacità $2k$, è conveniente descrivere la dinamica del bilancio tramite la variabile centrata

$$X_t(u, v) = \frac{b_e^{(t)}(v) - b_e^{(t)}(u)}{2} \in \{-k, \dots, k\},$$

che vale 0 quando il bilancio è perfettamente equilibrato, $+k$ quando v possiede l'intera capacità, e $-k$ quando u possiede l'intera capacità. Ogni transazione che attraversa il canale in una direzione sposta X_t di $+1$, ogni transazione nella direzione opposta lo sposta di -1 . Il canale è vuoto in una direzione se e solo se $|X_t| = k$.

Il comportamento di questa passeggiata aleatoria [3] dipende dal modello di traffico adottato:

- **Modello uniforme.** I pagamenti si distribuiscono simmetricamente tra le due direzioni del canale. X_t oscilla intorno allo zero e raggiunge le barriere $\pm k$ solo dopo un tempo elevato — dell'ordine di k^2 iterazioni per un singolo canale.
- **Modello con Merchant.** La presenza di nodi *Merchant* come destinazione privilegiata introduce una componente di deriva che inclina sistematicamente la passeggiata verso una delle barriere, riducendo il tempo al primo esaurimento.

2.7.2 Il tempo di fallimento τ

Introduciamo ora la metrica principale dell'analisi: il *tempo di fallimento*, indicato con τ .

Dato il processo aleatorio che descrive la sequenza di pagamenti sulla rete, viene dunque introdotto τ come il numero totale di transazioni completate prima che si verifichi il primo fallimento, ovvero il primo istante in cui almeno un canale è vuoto in una direzione:

$$\tau = \inf\{t \in \mathbb{N} : \exists e \in E \text{ con } b_{\min}(e) = 0\}. \quad (2.7.1)$$

La sfida analitica consiste nello stimare il valore medio di τ e nel capire come esso dipenda dalla topologia della rete, dalle capacità dei canali e dalla distribuzione del traffico. Nel modello uniforme, i risultati teorici di riferimento mostrano che τ cresce quadraticamente con la capacità k e con il numero di nodi n . L'introduzione dei nodi *Merchant* altera questa stima, introducendo una deriva che accelera il raggiungimento delle "barriere critiche" e riduce τ in modo proporzionale alla crescita dell'asimmetria del traffico.

2.8 Meccanismi di Ribilanciamento: Il JIT Rebalancing

Come già ampiamente discusso l'analisi dei flussi asimmetrici con nodi Merchant sottolinea un problema a livello strutturale delle reti PCN, ovvero la saturazione unidirezionale dei canali. Infatti i canali che collegano la rete ai nodi Merchant tendono a svuotarsi rapidamente del loro bilancio in entrata rendendo impossibile l'instradamento di ulteriori pagamenti verso il Merchant, nonostante la capacità totale dei canali rimane invariata.

Per contrastare questa "morte precoce" della rete, introduciamo il meccanismo di **Just-In-Time (JIT) Rebalancing**. A differenza di approcci precedenti come [Pickhardt Nowostawski, 2019] [4], questa strategia opera in maniera tale da affrontare il problema appena si presenta (just in time) e ottimizza la selezione del percorso basandosi sulla topologia locale del Merchant e sulla stabilità dei nodi intermedi.

2.8.1 Conservazione della liquidità locale

L'operazione si basa sulla creazione di un ciclo virtuale che coinvolge il Merchant m , il Client c e un terzo nodo w . Formalmente, sia $e = (c, m)$ l'arco con bilancio $b(c, m) = 0$. Il Merchant tenta di ripristinare la propria capacità di ricezione senza aggiungere altra liquidità dall'esterno, ma ridistribuendo quella esistente. Per un ciclo $C = (m \rightarrow c \rightarrow \dots \rightarrow w \rightarrow m)$, l'operazione di ammontare Δ comporta:

- **Aumento della capacità di ricezione:** $b(c, m) \leftarrow b(c, m) + \Delta$
- **Ricarica del canale più debole lato Merchant :** $b(m, w) \leftarrow b(m, w) + \Delta$

2.8.2 Selezione Ottimale del Canale di Uscita

Il punto critico della strategia è la scelta del nodo w da cui prelevare i fondi. Invece di una selezione casuale, l'algoritmo applica un criterio di minimizzazione del bilancio locale:

$$w_{opt} = \min_{w \in \text{Vicini}(m) \setminus \{c\}} (b(m, w))$$

Minimizzare $b(m, w)$ equivale a selezionare il canale in cui la liquidità è maggiormente sbilanciata verso il nodo remoto ($b(w, m)$ è massimo). Dato che l'ultimo arco del ciclo di rebalancing richiede che il nodo w invii fondi al Merchant, scegliere il canale con il valore $b(w, m)$ più alto massimizza drasticamente la probabilità che l'operazione vada a buon fine, riducendo il rischio di fallimento nell'ultimo spostamento (operazione più a rischio).

Dunque questa operazione è a somma zero per la liquidità totale del Merchant, ma ne massimizza l'utilità spostando i fondi dai canali passivi (lato merchant) ai canali attivi che erano bloccati.

Capitolo 3

Metodologia e Modello di Simulazione

In questo capitolo viene presentato in pseudocodice il lavoro sperimentale svolto per analizzare la resilienza della rete. L'algoritmo si concentra sull'impatto dei flussi asimmetrici dettati dalla probabilità che un nodo di tipo Merchant venga scelto. L'algoritmo prende in input un grafo non diretto, dove ad ogni arco vi è associata una capacità, ad ogni nodo un bilancio, inoltre abbiamo un sottoinsieme di nodi M i quali vengono scelti come destinazione a seconda di una probabilità α passata in input. Per prima cosa vengono impostati i bilanci per ogni arco al parametro k e viene inizializzato τ a 0, poi in un ciclo infinito ci scegliamo una sorgente per un pagamento e una destinazione secondo il bias impostato, tentando di instradare il pagamento lungo un cammino e aggiornando i bilanci locali degli archi coinvolti. Il processo prosegue iterativamente incrementando il contatore τ finché la liquidità nei canali non si sbilancia al punto da rendere impossibile la ricerca di un percorso valido; a quel punto, la simulazione termina restituendo il valore di τ come misura della stabilità del sistema

3.1 L'Algoritmo di Simulazione

Algorithm 1: PCN Failure Time Simulation

Input: Grafo $G = (V, E)$, Capacità k , Probabilità Merchant α , Lista Merchant M

Output: Tempo di fallimento τ

```
1 foreach arco  $\{u, v\} \in E$  do
2   |  $b_{uv} \leftarrow k$   $b_{vu} \leftarrow k$ 
3 end
4  $\tau \leftarrow 0$ 
5 while True do
6   |  $s \leftarrow \text{estrai\_uniforme}(V)$ 
7   |  $p \leftarrow \text{estrai\_uniforme}(0, 1)$ 
8   | if  $p < \alpha$  then
9     |  $d \leftarrow \text{estrai\_uniforme}(M \setminus \{s\})$  end
10    | else
11      |  $d \leftarrow \text{estrai\_uniforme}(V \setminus \{s\})$ 
12    | end
13    |  $P \leftarrow \text{FindPath}(G, s, d, \text{weight} : b > 0)$ 
14    | if  $P$  esiste then
15      | foreach arco  $(u, v) \in P$  do
16        |  $b_{uv} \leftarrow b_{uv} - 1$ 
17        |  $b_{vu} \leftarrow b_{vu} + 1$ 
18      | end
19      |  $\tau \leftarrow \tau + 1$ 
20    | end
21    | else
22      | return  $\tau$ 
23    | end
24  end
25
```

3.2 Configurazione del Sistema e Ipotesi Iniziali

3.2.1 Inizializzazione dei Bilanci e Stato di Equilibrio

In questo studio, ogni canale $e = \{u, v\}$ viene inizializzato in uno stato di **equilibrio**. Formalmente, data una capacità totale $C = 2k$, i bilanci iniziali al tempo $t = 0$ sono impostati come:

$$b_{uv}(0) = b_{vu}(0) = k \quad (3.2.1)$$

Questa scelta è dettata dalla necessità di lavorare in un ambiente "neutro". Partire da una configurazione bilanciata permette di:

1. **Isolare la variabile Drift:** Ogni sbilanciamento osservato durante la simulazione sarà dato dalla dinamica del traffico (parametro α) e non a una disparità di fondi già esistente.
2. **Analisi della Passeggiata Aleatoria:** Lo stato 0 (equilibrio) rappresenta l'origine della passeggiata aleatoria. Questo ci permette di misurare con precisione il tempo τ necessario affinché il processo stocastico colpisca le "barriere assorbenti" rappresentate dall'esaurimento della liquidità. ($b = 0$).

3.2.2 Quantizzazione dei Pagamenti

Si è scelto di modellare ogni transazione fissando la quota di pagamento $x = 1$. Questa semplificazione, coerente con i modelli teorici di riferimento, trasforma il problema della liquidità in un problema di *conteggio dei passi*. In questo modo, la capacità k non rappresenta solo un valore monetario, ma il numero massimo di transazioni che un canale può sostenere prima di fallire.

3.3 Modellazione del Traffico Asimmetrico

Definiamo un set di nodi $M \subset V$ come **Merchant**. Il traffico viene generato seguendo questa logica:

- **Sorgente (s):** Estratta uniformemente tra tutti i nodi (V). Rappresenta l'utente finale che vuole effettuare un pagamento.
- **Destinazione (d):** Regolata dal parametro di bias $\alpha \in [0, 1]$.
 - Con probabilità α , $d \in M$ (flusso verso i Merchant).
 - Con probabilità $1 - \alpha$, $d \in V \setminus \{s\}$ (flusso uniforme).

Algorithm 2: Rebalancing

Input: Lista canali esausti *failed*, matrice bilanci B , lista Merchant M , Grafo G

Output: Success (Bool), Arco del fallimento

```

1 foreach  $(u, v) \in failed$  do
2   if  $\min(B[u, v], B[v, u]) > 0$  then
3     continue           // Già risolto da rebalance precedente
4   end
5    $(c, m) \leftarrow \text{identifica\_client\_e\_merchant}(u, v, M)$ 
6   /* Fase 1: Selezione ottimale del canale di uscita (Outbound) */
7    $w_{opt} \leftarrow \arg \min_{w \in vicini} (B[m, w])$            // Canale più vuoto lato Merchant
8   /* Fase 2: Pathfinding Privacy-Preserving */
9    $G_{sub} \leftarrow G$  privo di altri nodi Merchant ( $M \setminus \{c, w_{opt}\}$ )
10   $P_{c\_w} \leftarrow \text{ShortestPath}(G_{sub}, c, w_{opt})$ 
11  /* Fase 3: Probing e Esecuzione */
12  if  $P_{c\_w}$  esiste and  $\text{VerificaLiquidità}(B, m, c, P_{c\_w}, w_{opt})$  then
13    // Aggiorna bilanci lungo il ciclo: m -> c -> ...
14    // -> w_{opt} -> m
15     $\text{EseguiSpostamentoFondi}(B, m, c, P_{c\_w}, w_{opt}, \text{amount} = 1)$ 
16  end
17  else
18    return False,  $(u, v)$            // Rebalancing fallito
19  end
20 end
21 return True, None

```

3.3.1 Descrizione dell'algoritmo di Rebalancing

L'algoritmo fornisce una strategia di riequilibrio proattiva guidata dai nodi Merchant, mirata a ripristinare la liquidità nei canali esausti identificati durante la fase di instradamento. A differenza di approcci generici, questa tecnica sfrutta la topologia locale del Merchant per creare cicli di riequilibrio. L'algoritmo si divide in 4 fasi principali:

1. **Filtro e identificazione dei ruoli:** L'algoritmo itera sulla lista dei canali falliti. Per ogni coppia (u,v) viene inizialmente verificato se un'operazione di rebalance precedente abbia già risolto l'esaurimento. Successivamente, tramite la funzione *identifica_client_e_merchant*, vengono distinti i ruoli: il nodo $m \in M$ rappresenta il Merchant che vuole ripristinare liquidità in entrata, mentre c è il client associato al canale esausto. Si assume che ogni arco fallito abbia almeno un endpoint in M . Questa ipotesi è giustificata dal fatto che, in presenza di forte bias, i canali che si esauriscono per primi sono strutturalmente quelli del cut-set dei Merchant. Un arco tra due nodi client potrebbe in linea di principio esaurirsi, ma con probabilità trascurabile rispetto agli archi incidenti ai Merchant.
2. **Selezione Ottimale del Canale di Uscita:** per bilanciare il portafoglio dei canali del Merchant, l'algoritmo non sceglie un vicino casuale, ma sceglie un nodo w_{opt} tra i vicini di m che presenta il bilancio locale minimo. L'obiettivo è prelevare fondi da un canale con scarsa liquidità lato Merchant per spostarli verso il canale svuotato con c .
3. **Costruzione della Sottorete:** Per massimizzare le probabilità di successo del rebalancing, l'algoritmo genera un grafo ausiliario senza altri nodi Merchant eccetto quelli necessari all'operazione. Questa scelta è dettata da una necessità: I Merchant sono canali soggetti a frequente saturazione. Escludendo gli altri Merchant, l'algoritmo evita di instradare il flusso di riequilibrio attraverso canali critici o potenzialmente esausti, preferendo percorsi che attraversano nodi standard. Questo riduce drasticamente il rischio di incontrare colli di bottiglia.

4. **Ricerca del Cammino e Attuazione:** Viene calcolato il cammino minimo tra il client c e w_{opt} all'interno del grafo ausiliario senza merchant. Se questo percorso esiste, l'algoritmo esegue una funzione di verifica della liquidità lungo l'intero ciclo (che simula una fase di probing nella realtà). In caso positivo, viene effettuato uno spostamento dei fondi (di un unità) che ripristina la capacità operativa del canale originale.

Algorithm 3: PCN Simulation con Rebalancing

Input: Grafo $G = (V, E)$, Capacità k , Probabilità Merchant α , Lista Merchant M

Output: Tempo di fallimento τ

```

1 foreach arco  $\{u, v\} \in E$  do
2   |  $b_{uv} \leftarrow k$   $b_{vu} \leftarrow k$ 
3 end
4  $\tau \leftarrow 0$ 
5 while True do
6   |  $s \leftarrow \text{estrai\_uniforme}(V)$ 
7   |  $p \leftarrow \text{estrai\_uniforme}(0, 1)$ 
8   | if  $p < \alpha$  then
9     |  $d \leftarrow \text{estrai\_uniforme}(M \setminus \{s\})$  end
10    | else
11      |  $d \leftarrow \text{estrai\_uniforme}(V \setminus \{s\})$ 
12    | end
13    |  $P \leftarrow \text{FindPath}(G, s, d, \text{weight} : b > 0)$ 
14    | if  $P$  esiste then
15      | foreach arco  $(u, v) \in P$  do
16        |  $b_{uv} \leftarrow b_{uv} - 1$ 
17        |  $b_{vu} \leftarrow b_{vu} + 1$ 
18      | end
19      |  $\tau \leftarrow \tau + 1$ 
20      |  $\text{/* Tentativo di Rebalancing se un canale è$ 
21        |  $\text{esausto} \text{ */}$ 
22      | if  $\exists (u, v) \in P : b_{uv} = 0 \text{ or } b_{vu} = 0$  then
23        |  $\text{success} \leftarrow \text{AttemptRebalance}(G, M, b)$ 
24        | if  $\text{!success}$  then
25          | return  $\tau$ 
26        | end
27      | end
28    | end
29    | return  $\tau$ 
30  end

```

31

3.3.2 Descrizione dell'algoritmo: PCN Simulation con Rebalancing

L'algoritmo 3 rappresenta il nucleo del simulatore sviluppato e descrive l'intero ciclo di vita della rete, integrando la logica di ribilanciamento. L'obiettivo è quantificare il tempo di fallimento τ in un ambiente dinamico dove la liquidità viene costantemente ridistribuita. Le differenze dall'algoritmo iniziale sono le seguenti:

1. **Check del Fallimento e Innesco del Rebalancing:** Dopo ogni transazione, il sistema verifica se qualche canale ha raggiunto il bilancio zero. In assenza di rebalancing questo comporterebbe la fine della simulazione. Con l'attivazione del rebalancing, invece, il sistema identifica ogni arco esaurito e tenta il ribilanciamento.
2. **Integrazione della strategia di Rebalancing:** Il cuore della differenza si trova nella funzione *Rebalancing*.
3. **Verifica Instradamento:** Una volta individuato un potenziale ciclo di ribilanciamento, il sistema deve verificarne la fattibilità reale: In una rete reale, il nodo Merchant invierebbe un pacchetto HTLC di prova lungo il percorso scelto, ricevendo una risposta se è possibile effettuare un pagamento o meno, mentre per semplificazione nel nostro pseudocodice questa fase viene eseguita tramite una verifica diretta sulla matrice dei bilanci reali. Sebbene il simulatore acceda a dati privati, il nodo Merchant non ne è a conoscenza, rispettando così i vincoli di privacy della rete.
4. **Criterio di Terminazione τ :** In questo modello, il tempo di fallimento τ è strettamente legato alla capacità di riequilibrio. La simulazione non termina solo se non esiste un cammino per il pagamento (s,d), ma anche se la manovra di rebalancing fallisce. Questo rende τ uno stress-test diretto per la strategia di gestione della liquidità dei Merchant: se il Merchant non riesce a svuotare i propri canali saturi verso il Client, la simulazione si interrompe.

Capitolo 4

Simulazioni e Risultati

In questo capitolo vengono esposti i risultati ottenuti attraverso le simulazioni. L'obiettivo principale è comprendere l'impatto dei flussi di pagamento con nodi Merchant rispetto la stabilità della rete in termini della distribuzione della liquidità nei vari canali. L'impatto più visibile sarà senz'altro il confronto con il modello uniforme in quanto come spiegato nei capitoli precedenti, ci aspettiamo un crollo vertiginoso della rete in quanto i canali che fanno parte del cut-set dei nodi Merchant sono un collo di bottiglia evidente.

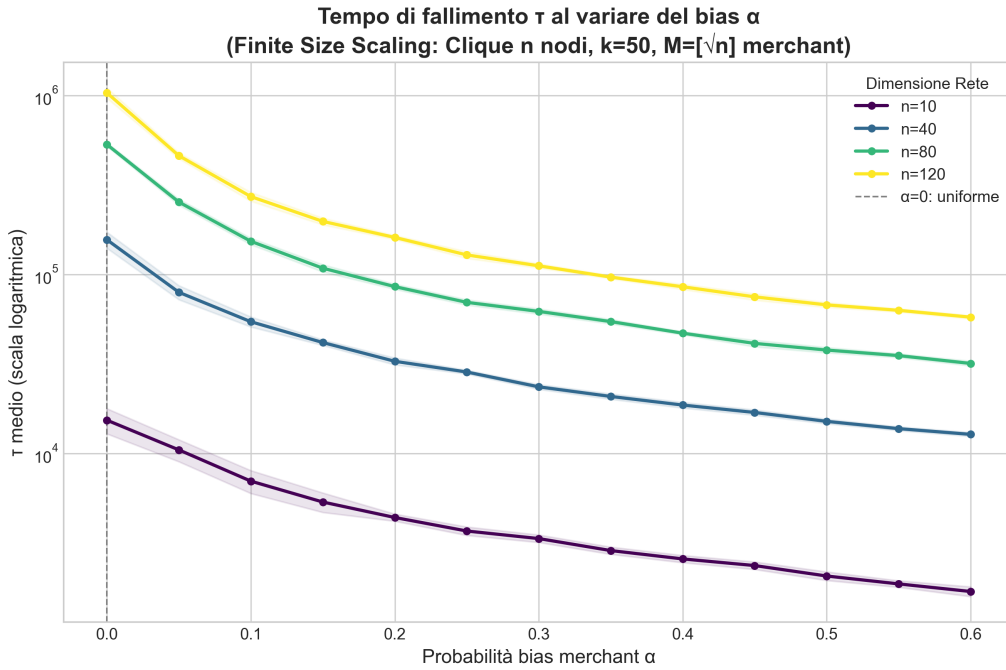
4.1 Ambiente di Sviluppo

Per realizzare il simulatore è stato scelto il linguaggio di programmazione Python, scelto soprattutto per la sua semplicità e la varietà di librerie utili per questo progetto quali librerie per la generazione automatica di grafi (NetworkX) e per la manipolazione dei dati da mettere a grafico (matplotlib).

4.2 Implementazione

Il cuore dell'esperimento è l'implementazione dello script basato sull'algoritmo 1 (PCN Failure Time Simulation) presentato nel Capitolo 3.

4.3 Grafico τ in funzione di α

Figura 4.1: τ in funzione di α

Per questo primo grafico le simulazioni effettuate sono state fatte nel seguente modo: per prima cosa sono stati lavorati solo topologie Clique, per ogni configurazione sono state effettuate circa 20 run indipendenti e preso il valore medio di τ , ogni configurazione ha $k = 50$ prendendo Clique di diverse dimensioni. Nella figura 4.1 viene analizzata come la dimensione della rete (il numero di nodi n), influenzi il tempo di vita della lightning network andando a diminuire drasticamente il numero di transazioni dopo il primo fallimento τ . In effetti questo grafico evidenzia una netta transizione tra il regime uniforme (con bias $\alpha = 0$) e regime non uniforme (con bias $\alpha > 0$). In assenza di bias la lightning network beneficia dell'aumento dei nodi mostrando una crescita di τ quadratica come previsto. Al contrario, nel regime Merchant ($\alpha > 0$) si osserva un appiattimento delle curve, in quanto la presenza di questi nodi che aumentano il flusso dei pagamenti verso loro, riescono a interrompere

questo meccanismo di auto bilanciamento statisco rendendo l'aggiunta di nuovi nodi n piano piano sempre meno efficace nel posticipare il fallimento della lightning network.

4.4 Grafico τ in funzione di k in scala log log

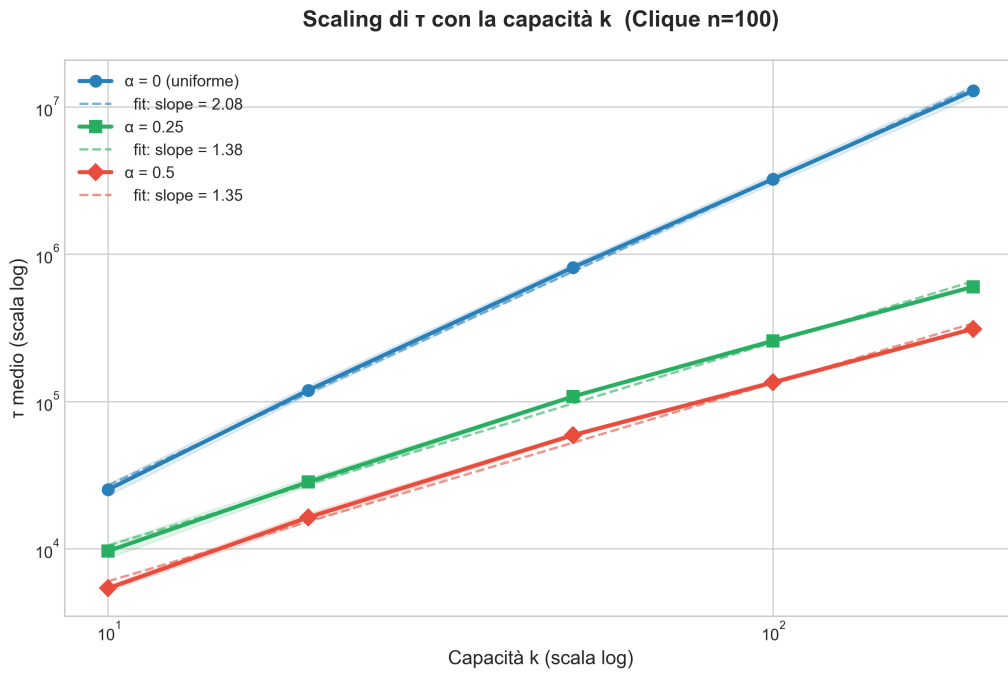


Figura 4.2: τ in funzione di k , prendendo tre valori di α

L'analisi della stabilità in funzione della capacità dei canale k rileva una transizione nel comportamento della rete. In un grafico log-log la pendenza delle rette rappresenta l'esponente della relazione $\tau \propto k^\beta$, Permettendo una distinzione tra i regimi che stiamo analizzando: Nel regime uniforme, come possiamo vedere e come ci aspettavamo lo slope misurato è pari 2.08, un valore che conferma le previsioni teoriche (considerando che la simulazione è stata fatta per un n molto basso). Mentre nel regime Merchant, introducendo questo bias α otteniamo valori compresi tra 1.38 e 1.35, questo passaggio a un regime sub-quadratico indica che la dinamica del sistema non è più

stocastica, ma dominata da un componente deterministico verso i Merchant. In conclusione, questi dati dimostrano che in presenza di traffico asimmetrico, aumentare la capacità dei canali non offre gli stessi rendimenti del caso uniforme.

4.5 Analisi del trasferimento di ricchezza e del Sink Effect

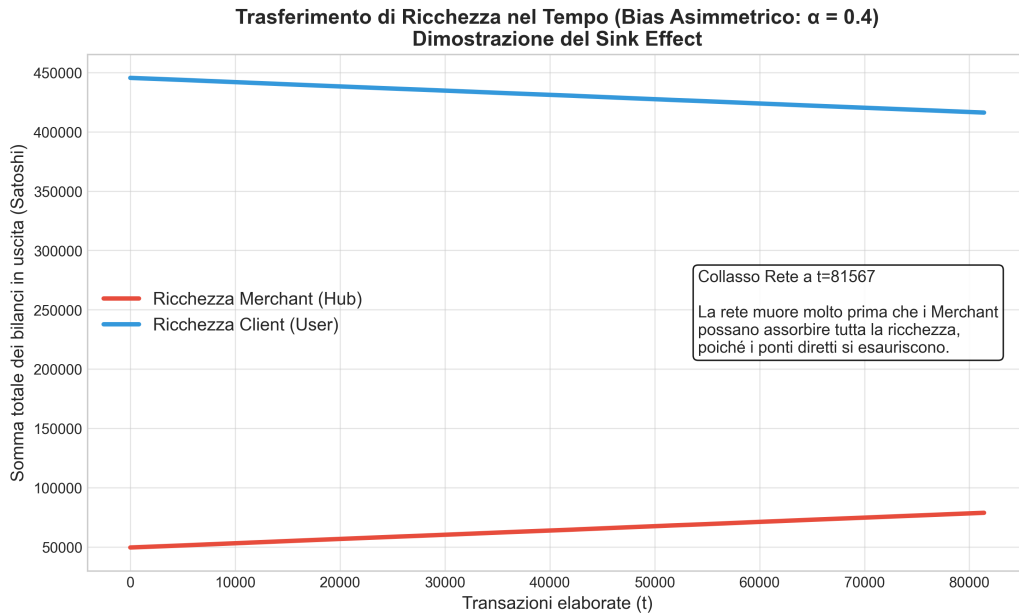


Figura 4.3: Evoluzione temporale della somma dei bilanci in uscita suddivisa tra nodi client e nodi Merchant

Il grafico fornisce una prova del Sink Effect, ovvero il fenomeno per cui i nodi Merchant agiscono come assorbitori di liquidità all'interno del sistema. Infatti con un bias verso i Merchant, si osserva un trasferimento di ricchezza costante e lineare. L'aspetto più critico evidenziato dalla simulazione è che il collasso della rete avviene per $\tau = 81567$, un istante in cui i nodi Client possiedono ancora la maggioranza della ricchezza totale del sistema. Questo conferma l'ipotesi centrale della tesi: la rete non fallisce per mancanza di fondi com-

4.6. FREQUENZA DI FALLIMENTO DEGLI ARCHI IN FUNZIONE DI BETWEENNESS CENTRALITY E DEL MERCHANT BIAS

plessivi, ma per l'esaurimento prematuro dei bilanci locali nel cut-set del Merchant. Dunque i merchant diventano irraggiungibili non perché non ci siano più bitcoin nella rete, ma perché i ponti diretti che permettono di versare fondi verso di loro sono stati completamente svuotati. Questo grafico sottolinea come la stabilità della Lightning Network sia una proprietà della distribuzione della liquidità piuttosto che della sua quantità totale

4.6 Frequenza di fallimento degli archi in funzione di Betweenness Centrality e del Merchant bias

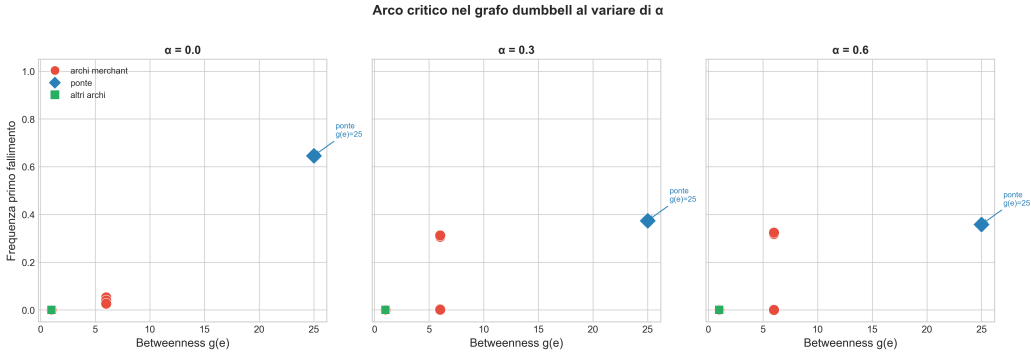


Figura 4.4: Grafico che mette a confronto due tipi diversi di fallimento

Questo grafico nasce dalla necessità di individuare quale tra le principali cause di fallimento sono le più critiche tra Betweenness Centrality e bias Merchant. Per effettuare questo esperimento è stata scelta una topologia specifica, il grafo Dumbbell, essenzialmente è un grafo composto da due componenti connesse (Clique) unite da un singolo arco centrale, chiamato anche ponte. Questa struttura è ideale per isolare i due colli di bottiglia delle lightning network. L'analisi di questo scenario ha permesso di dimostrare che il fallimento della rete non è un evento casuale ma colpisce gli archi più coinvolti dai flussi di pagamento. Come mostrato nel grafico, per il traffico uniforme, il ponte è l'unico arco critico, fallendo a causa della sua posizione struttu-

4.7. GRAFICO τ IN FUNZIONE DI k NELLA LIGHTNING NETWORK

le. Aumentando il bias verso il merchant, possiamo notare che la criticità si sposta, mentre il ponte rimane rilevante, gli archi incidenti al Merchant (evidenziati in rosso) iniziano a mostrare una frequenza di fallimento crescente, nonostante la loro betweenness centrality sia molto più bassa rispetto l'arco ponte. Questo risultato indica che l'introduzione di un Merchant distorce la gerarchia di importanza degli archi: la centralità topologica non è più l'unico indice di fallimento.

4.7 Grafico τ in funzione di k Nella Lightning Network

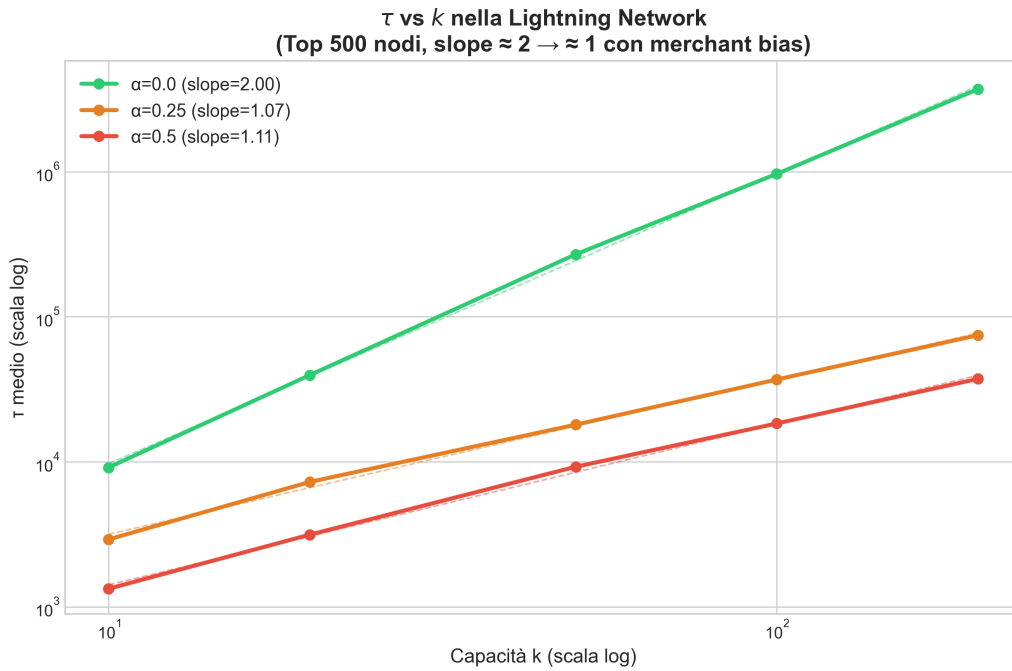


Figura 4.5: Risultati di τ in funzione di k su 500 nodi selezionati nello snapshot lightning network

Le analisi sono state estese alla topologia reale della Lightning Network estrapolata da uno snapshot datato 23 marzo 2026. In fase di elaborazione del file

4.7. GRAFICO τ IN FUNZIONE DI k NELLA LIGHTNING NETWORK

json, il grafo è stato ripulito escludendo i canali privi di policy di routing bidirezionale o marcati come disabilitati. Per gli esperimenti è stato estratto un sottografo fortemente connesso costituito dai top 500 nodi ordinati per grado. all'interno di questa rete, l'insieme dei nodi Merchant è stato scelto selezionando i nodi con il grado più alto; Sono stati selezionati 22 Merchant in quanto come deciso anche per altre simulazioni il numero di Merchant è pari a $\lfloor \sqrt{N} \rfloor$. L'analisi di τ in funzione delle capacità k rivela una profonda transizione nel comportamento della rete reale. In un regime di traffico uniforme, la Lightning Network manifesta la sua massima efficienza, infatti andando ad analizzare la pendenza della retta in scala log-log otteniamo 2.00, confermando che il tempo di vita della rete cresce quadraticamente rispetto alla liquidità dei canali. Tuttavia, l'introduzione di un bias verso i nodi Merchant rompe questa simmetria, facendo precipitare la pendenza verso valori prossimi ad 1 (1.07 e 1.11), Questo passaggio da regime quadratico a lineare indica che in presenza di questi nodi Merchant la liquidità non circola più liberamente ma viene "consumata". In questo scenario aumentare la capacità dei canali non offre più gli stessi rendimenti, trasformando la robustezza della rete in una fragilità a livello strutturale dove ogni incremento di k si trasforma in un guadagno di tempo irrisorio.

4.8 Effetti del Rebalance nella lightning Network

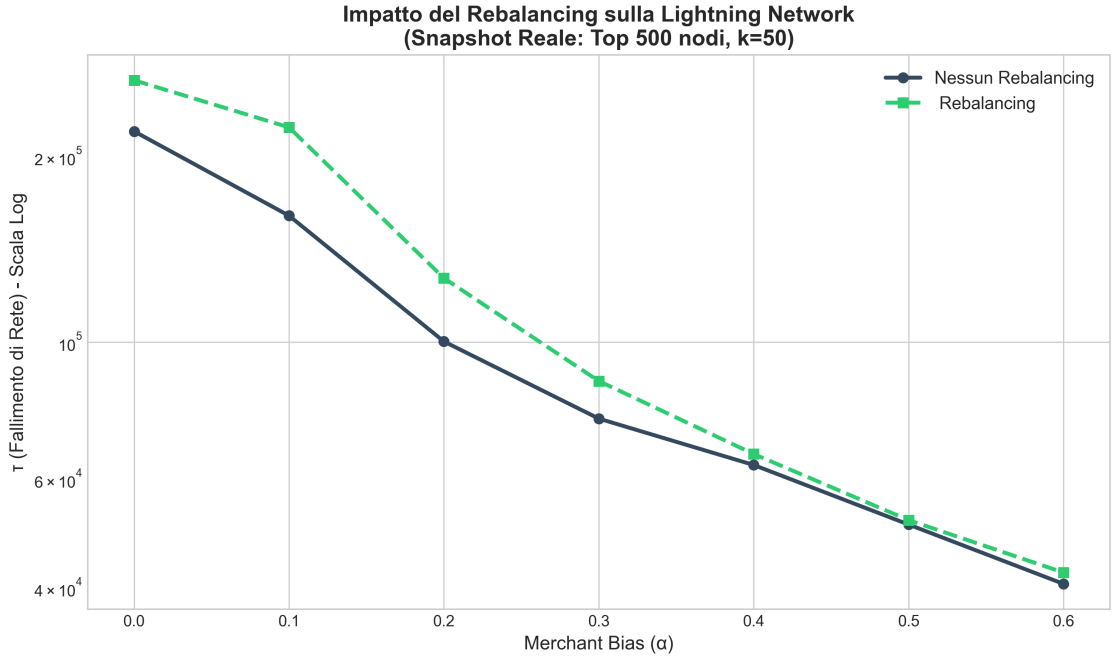


Figura 4.6: Confronto del tempo di vita della lightning network con e senza Rebalancing

I dati ottenuti dalla simulazione effettuata su uno snapshot reale della Lightning Network (top 500 nodi, con bilanci singoli $k = 50$), confermano l'efficacia della strategia di riequilibrio proposta. Il grafico mette in relazione il tempo di fallimento τ con il bias α , ovvero la probabilità che una transazione sia diretta verso un nodo Merchant.

1. **Estensione della Vita Utile della Rete** Il dato più evidente è che la curva "rebalancing" si posiziona costantemente al di sopra della curva "Nessun Rebalancing". Questo dimostra che la capacità dei Merchant di gestire attivamente la propria liquidità permette alla rete di sostenere un volume di traffico significativamente maggiore prima di incorrere in un fallimento strutturale.

- Per valori di $\alpha = 0$, l'incremento di τ è massimo, evidenziando come il rebalancing sia efficace anche a correggere degli sbilanciamenti naturali della rete.
2. **Sensibilità al Merchant bias** Si osserva un decadimento quasi esponenziale del tempo di fallimento all'aumentare di α . Man mano che il traffico si concentra sui Merchant, la velocità con cui i canali si svuotano supera la capacità della rete di ricaricarsi. Quando il flusso è fortemente asimmetrico, la liquidità tende ad accumularsi staticamente presso i Merchant, rendendo i cicli di riequilibrio sempre più difficili da completare per mancanza di percorsi alternativi.
 3. **Rendimenti Decrescenti** Un aspetto interessante dei risultati è la convergenza delle due curve per valori elevati di α (quando oltre il 50% dei pagamenti è diretto verso i Merchant). Infatti in questo caso la liquidità non è solo distribuita male, ma è esaurita lungo la maggior parte dei percorsi. Dato che la nostra strategia esclude altri merchant per evitare canali critici, l'assenza di percorsi alternativi tra i nodi client diventa il collo di bottiglia che annulla i benefici della manovra di rebalancing.

Capitolo 5

Conclusioni

Questo lavoro ha analizzato la stabilità della liquidità nella Lightning Network in presenza di flussi di pagamento asimmetrici, con l'obiettivo di quantificare l'effetto dei nodi Merchant sul tempo di primo fallimento τ e di valutare l'efficacia di un meccanismo di ribilanciamento come strumento di contenimento. Il punto di partenza è stato il modello a traffico uniforme, in cui i nodi sorgente e destinazione di ogni pagamento vengono scelte uniformemente a caso tra tutti i nodi della rete. In questo regime, i risultati teorici di riferimento prevedono una crescita quadratica di τ rispetto alla capacità k dei canali, un comportamento confermato dalle simulazioni con una pendenza misurata di 2.08 su topologia Clique e di 2.00 su snapshot reale della Lightning Network. Questi risultati validano il simulatore implementato e costituiscono la base rispetto alla quale valutare l'effetto dell'asimmetria. L'introduzione del parametro di bias α che regola la probabilità che la destinazione di un pagamento sia un nodo Merchant, ha prodotto alcuni effetti coerenti sulle topologie testate. Nel modello a traffico asimmetrico, la crescita di τ in funzione di k degrada a un regime sub-quadratico, con esponenti compresi tra 1.35 e 1.38 su topologia Clique e tra 1.07 e 1.11 sullo snapshot reale. Questo passaggio indica che in presenza di nodi Merchant la liquidità non circola più liberamente ma viene consumata in modo unidirezionale: aumentare quindi la capacità dei canali offre rendimenti decrescenti, e la robustezza della rete si trasforma in una debolezza strutturale. Il motivo di questo comportamento è identificabile nel riempimento del cut-set critico formato dai canali incidenti ai nodi Merchant.

Il meccanismo di Rebalancing introdotto nel Capitolo 2, ha confermato l'efficacia della gestione della liquidità nel mitigare lo sbilanciamento dei canali indotto dal traffico asimmetrico. Contrariamente a un modello passivo, i risultati sperimentali dimostrano che il riequilibrio permette di estendere sensibilmente la vita utile della rete, con una curva che si posiziona costantemente al di sopra dello scenario privo di interventi. Tuttavia, l'efficacia

di tale manovra è strettamente legata al parametro di bias α : al crescere dell'asimmetria, i rendimenti diventano decrescenti fino a convergere per valori di $\alpha 0.5$. Questo fenomeno è riconducibile alla saturazione dei percorsi alternativi tra i nodi client, che diventano il vero collo di bottiglia strutturale e impediscono il completamento dei cicli di riequilibrio. In conclusione, sebbene il rebalancing si dimostri uno strumento potente per correggere gli sbilanciamenti, la sua capacità di compensare interamente il "Sink Effect" verso i Merchant resta vincolata alla connettività residua della topologia circostante.

5.1 Limiti e sviluppi futuri

Il modello adottato presenta semplificazioni che aprono a possibili sviluppi futuri. In primis, ogni transazione è stata modellata con importo unitario; Introducendo una distribuzione degli importi più realistica potrebbe alterare la dinamica di sbilanciamento. Inoltre il parametro α è stato trattato come costante nel tempo, mentre nella realtà l'intensità del traffico verso i Merchant varia in base a molti fattori. Un modello con bias variabile nel tempo sarebbe più vicino alla realtà della Lightning Network. Un'altra idea sarebbe lo sviluppo di una strategia di ribilanciamento proattiva (magari basata su soglie di liquidità) piuttosto che reattiva). Sarebbe interessante effettuare uno studio integrando una terza classificazione dei nodi: i nodi banca. In uno scenario dove la liquidità tende ad accumularsi staticamente presso i Merchant rendendo i canali unidirezionali, questi nodi agirebbero come regolatori della rete. Il loro compito principale sarebbe quello di prelevare attivamente la liquidità in eccesso dai nodi Merchant per ridistribuirla strategicamente verso nodi Client. In questa maniera il rebalancing si trasformerebbe da una manovra reattiva a un vero e proprio mantenimento della rete. Lo studio di questa terza classe di nodi aprirebbe la strada all'analisi di nuovi algoritmi di instradamento.

In conclusione, i risultati di questa tesi dimostrano che la presenza di nodi Merchant nella Lightning Network costituisce una sfida strutturale che non può essere risolta semplicemente aumentando la capacità dei canali. Il

ribilanciamento rappresenta una direzione interessante, ma la sua efficacia dipende in modo critico dalla topologia della rete e dalla disponibilità di percorsi alternativi con capacità sufficiente.

Elenco delle figure

4.1	τ in funzione di α	24
4.2	τ in funzione di k , prendendo tre valori di α	25
4.3	Evoluzione temporale della somma dei bilanci in uscita suddivisa tra nodi client e nodi Merchant	26
4.4	Grafico che mette a confronto due tipi diversi di fallimento . .	27
4.5	Risultati di τ in funzione di k su 500 nodi selezionati nello snapshot lightning network	28
4.6	Confronto del tempo di vita della lightning network con e senza Rebalancing	30

Elenco degli algoritmi

1	PCN Failure Time Simulation	16
2	Rebalancing	18
3	PCN Simulation con Rebalancing	21

Bibliografia

- [1] J. Poon and T. Dryja, “The bitcoin lightning network: Scalable off-chain instant payments,” Technical Report, Tech. Rep., 2016. [Online]. Available: <https://lightning.network/lightning-network-paper.pdf>
- [2] T. E. Abedesselam, F. Giacomelli, F. Pasquale, and M. Salvi, “Payment-failure times for random lightning paths,” in *2025 7th Conference on Blockchain Research amp; Applications for Innovative Networks and Services (BRAINS)*. IEEE, Nov. 2025, p. 1–10. [Online]. Available: <http://dx.doi.org/10.1109/BRAINS67003.2025.11302927>
- [3] L. László, “Random walks on graphs: A survey, combinatorics, paul erdos is eighty,” *Bolyai Soc. Math. Stud.*, vol. 2, 01 1993.
- [4] R. Pickhardt and M. Nowostawski, “Imbalance measure and proactive channel rebalancing algorithm for the lightning network,” 2019. [Online]. Available: <https://arxiv.org/abs/1912.09555>